

Trivy escáner de vulnerabilidades en Docker



Índice

Introducción.....	3
Conceptos clave de seguridad.....	4
Riesgos en contenedores:.....	4
Buenas prácticas:.....	4
Escaneo de vulnerabilidades.....	4
¿Qué es Trivy?.....	5
Características principales.....	5
Instalación y configuración de Trivy.....	6
Requisitos previos.....	6
Instalación.....	8
Actualizar paquetes del sistema.....	8
Herramientas para descargar archivos desde internet.....	8
Descargar trivy.....	9
Actualizar la base de datos de vulnerabilidades.....	10
Caso práctico: escanear una imagen vulnerable y una limpia.....	10
Imagen vulnerable:.....	10
Imagen no vulnerable:.....	13
Integración de Trivy en CI/CD.....	14
Conclusiones.....	22

Introducción

Los contenedores se han vuelto fundamentales en el desarrollo porque permiten ejecutar aplicaciones de manera aislada y eficiente.

Docker, la plataforma más utilizada para contenedores, facilita la creación y despliegue de estos entornos, pero no elimina los riesgos de seguridad.

Cada imagen puede contener vulnerabilidades en el sistema operativo o en las dependencias, configuraciones inseguras o paquetes desactualizados. Si no se detectan a tiempo, estas fallas pueden ser explotadas, provocando ataques, filtración de datos o fallos en producción.

Para mitigar estos riesgos, es esencial analizar las imágenes antes de utilizarlas en entornos productivos. Ahí es donde entran los escáneres de vulnerabilidades como Trivy entre otros, que permiten identificar de manera rápida y precisa problemas de seguridad en contenedores.

El objetivo de este trabajo es explorar Trivy: cómo instalarlo, configurarlo y usarlo para escanear imágenes Docker, interpretar los resultados y aplicar correcciones a las vulnerabilidades encontradas. Además, se abordará brevemente cómo integrarlo en pipelines de CI/CD, automatizando la seguridad de manera constante.

Conceptos clave de seguridad

La seguridad en contenedores es crítica, ya que cualquier vulnerabilidad en una imagen o dependencia puede comprometer todo el sistema.

Analizar y corregir fallos antes del despliegue es fundamental, usando buenas prácticas y herramientas especializadas que permitan un control de riesgos.

Riesgos en contenedores:

- Imágenes base desactualizadas con CVEs conocidas.
- Dependencias inseguras o sin parchear.
- Configuraciones inseguras: ejecución como root, puertos abiertos.
- Impacto: explotación remota, filtración de datos.

Buenas prácticas:

- Mantener imágenes y paquetes actualizados.
- Reducir software instalado al mínimo necesario.
- Evitar privilegios de root.
- Escaneo regular de imágenes con herramientas especializadas.

Escaneo de vulnerabilidades

- Detecta fallos antes de desplegar.
- Se integra en CI/CD para automatizar la seguridad.
- Herramientas principales: Trivy, Clair, Gype, Anchore, Snyk.

¿Qué es Trivy?

Es un escáner de vulnerabilidades de [código abierto](#) desarrollado por [Aqua Security](#).

Diseñado para analizar imágenes Docker, sistemas de archivos y repositorios de código en busca de fallos de seguridad.

Detecta vulnerabilidades conocidas (CVE) en sistemas operativos y dependencias de software, así como configuraciones inseguras.



Características principales

1. Rápido y fácil de usar, con resultados claros y detallados.
2. Compatibilidad amplia: soporta imágenes basadas en Alpine, Debian, Ubuntu, RHEL, entre otras.
3. Integración con CI/CD: permite automatizar
4. Escaneos en pipelines de desarrollo.
5. Mantenimiento activo: actualiza su base de datos de vulnerabilidades de manera continua.

Instalación y configuración de Trivy

Veremos cómo instalar Trivy, actualizar su base de datos y dejarlo listo para escanear imágenes Docker.

Requisitos previos

- Sistema operativo: Linux , macOS o Windows.
- Docker instalado y funcionando.
- Conexión a internet para descargar Trivy y actualizar su base de datos.

En mi caso, estaré usando [WSL con Ubuntu en Windows](#), ya que facilita y simplifica el uso de contenedores ,en mi opinión es más cómodo, pero es valido cualquier SO en máquina real o una MV.

```

adrian@DESKTOP-Q6JVA29: ~
Welcome to Ubuntu 24.04.2 LTS (GNU/Linux 5.15.167.4-microsoft-standard-WSL2 x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:        https://ubuntu.com/pro

System information as of Sun Oct 12 18:03:06 CEST 2025

System load:  0.24               Processes:    70
Usage of /:   1.8% of 1006.85GB  Users logged in: 0
Memory usage: 5%                IPv4 address for eth0: 172.20.115.84
Swap usage:   0%

 * Strictly confined Kubernetes makes edge and IoT secure. Learn how MicroK8s
just raised the bar for easy, resilient and secure K8s cluster deployment.
https://ubuntu.com/engage/secure-kubernetes-at-the-edge

This message is shown once a day. To disable it please create the
/home/adrian/.hushlogin file.
adrian@DESKTOP-Q6JVA29:~$ neofetch
      OS: Ubuntu 24.04.2 LTS on Windows 10 x86_64
      Kernel: 5.15.167.4-microsoft-standard-WSL2
      Uptime: 54 secs
      Packages: 812 (dpkg)
      Shell: bash 5.2.21
      Theme: Adwaita [GTK3]
      Icons: Adwaita [GTK3]
      Terminal: Relay(897)
      CPU: Intel i5-10400 (12) @ 2.904GHz
      GPU: 2720:00:00.0 Microsoft Corporation Basic Render Driver
      Memory: 858MiB / 15957MiB
  
```

1: Sistema Operativo (Ubuntu 24.04LTS en WSL)

```
$ neofetch
```



```
adrian@DESKTOP-Q6JVA29: ~$ docker --version
Docker version 20.10.7, build 20.10.7-0ubuntu1~20.04.1
adrian@DESKTOP-Q6JVA29: ~$ sudo docker run hello-world
Unable to find image 'hello-world:latest' locally
latest: Pulling from library/hello-world
17aec7bb9d7: Pull complete
Digest: sha256:54e66cc1dd1fcb1c3c58bd0017914dbed0701e2d8c74d9262e26bd9cc1642d31
Status: Downloaded newer image for hello-world:latest

Hello from Docker!
This message shows that your installation appears to be working correctly.

To generate this message, Docker took the following steps:
1. The Docker client contacted the Docker daemon.
2. The Docker daemon pulled the "hello-world" image from the Docker Hub.
   (amd64)
3. The Docker daemon created a new container from that image which runs the
   executable that produces the output you are currently reading.
4. The Docker daemon streamed that output to the Docker client, which sent it
   to your terminal.

To try something more ambitious, you can run an Ubuntu container with:
$ docker run -it ubuntu bash

Share images, automate workflows, and more with a free Docker ID:
https://hub.docker.com/

For more examples and ideas, visit:
https://docs.docker.com/get-started/

adrian@DESKTOP-Q6JVA29: ~$
```

```
$ docker --version
```

```
$ sudo docker run hello-world
```

```
adrian@DESKTOP-Q6JVA29: ~$ ping google.es
PING google.es (172.217.168.163) 56(84) bytes of data:
64 bytes from mad07s10-in-f3.1e100.net (172.217.168.163): icmp_seq=1 ttl=117 time=21.6 ms
64 bytes from mad07s10-in-f3.1e100.net (172.217.168.163): icmp_seq=2 ttl=117 time=21.5 ms
64 bytes from mad07s10-in-f3.1e100.net (172.217.168.163): icmp_seq=3 ttl=117 time=21.5 ms
64 bytes from mad07s10-in-f3.1e100.net (172.217.168.163): icmp_seq=4 ttl=117 time=21.5 ms
^C
--- google.es ping statistics ---
4 packets transmitted, 4 received, 0% packet loss, time 2998ms
rtt min/avg/max/mdev = 21.464/21.519/21.578/0.041 ms
adrian@DESKTOP-Q6JVA29: ~$
```

2: Conexión a Internet

```
$ ping google.es
```

Instalación

En este apartado se veremos cómo instalar Trivy en WSL/Ubuntu.

Actualizar paquetes del sistema

```

adrian@DESKTOP-Q6JVA29:~$ sudo apt update
[sudo] password for adrian:
Hit:1 http://security.ubuntu.com/ubuntu noble-security InRelease
Hit:2 https://deb.nodesource.com/node_18.x nodistro InRelease
Hit:3 http://archive.ubuntu.com/ubuntu noble InRelease
Hit:4 http://archive.ubuntu.com/ubuntu noble-updates InRelease
Hit:5 http://archive.ubuntu.com/ubuntu noble-backports InRelease
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
1 package can be upgraded. Run 'apt list --upgradable' to see it.
adrian@DESKTOP-Q6JVA29:~$ sudo apt upgrade -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
Calculating upgrade... Done
The following packages were automatically installed and are no longer required:
  libdrm-nouveau2 libdrm-radeon1 libgl1-amber-dri libglapi-mesa libllvm17t64 libllvm19 libxcb-dri2-0
Use 'sudo apt autoremove' to remove them.
Get more security updates through Ubuntu Pro with 'esm-apps' enabled:
  php-symfony-config libmagickcore-6.q16-7t64 php-symfony-dependency-injection
  imagemagick libmagickcore-6.q16-7-extra php-symfony-cache
  php-symfony-var-exporter php-twig imagemagick-6.q16
  php-symfony-expression-language imagemagick-6-common
  libmagickwand-6.q16-7t64 php-symfony-filesystem
Learn more about Ubuntu Pro at https://ubuntu.com/pro
The following packages have been kept back:
  libgl1-amber-dri
0 upgraded, 0 newly installed, 0 to remove and 1 not upgraded.
adrian@DESKTOP-Q6JVA29:~$

```

3: Actualización paquetes

```

$ sudo apt update
$ sudo apt upgrade -y

```

Herramientas para descargar archivos desde internet

```

adrian@DESKTOP-Q6JVA29:~$ sudo apt install wget curl -y
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
wget is already the newest version (1.21.4-1ubuntu4.1).
wget set to manually installed.
curl is already the newest version (8.5.0-2ubuntu10.6).
The following packages were automatically installed and are no longer required:
  libdrm-nouveau2 libdrm-radeon1 libgl1-amber-dri libglapi-mesa libllvm17t64 libllvm19 libxcb-dri2-0
Use 'sudo apt autoremove' to remove them.
0 upgraded, 0 newly installed, 0 to remove and 1 not upgraded.
adrian@DESKTOP-Q6JVA29:~$

```

4: get y curl, herramientas para descargar archivos desde internet

```

sudo apt install wget curl -y
# Instala wget y curl, herramientas para descargar archivos desde internet

```


[illegible]

```
$ wget https://github.com/aquasecurity/trivy/releases/download/v0.67.2/trivy_0.67.2_Linux-64bit.deb
$ sudo dpkg -i trivy_0.67.2 Linux-64bit.deb
```

```
trivy --version    # Comprueba que Trivy se instaló mostrando su versión
```

Actualizar la base de datos de vulnerabilidades

```
adrian@DESKTOP-Q6JVA29: ~$ trivy image --download-db-only
2025-10-12T19:26:52+02:00 INFO [vulndb] Need to update DB
2025-10-12T19:26:52+02:00 INFO [vulndb] Downloading vulnerability DB...
2025-10-12T19:26:52+02:00 INFO [vulndb] Downloading artifact... repo="mirror.gcr.io/aquasec/trivy-db:2"
72.58 MiB / 72.58 MiB [-----] 100.00% 20.25 MiB p/s 3.8s
2025-10-12T19:26:54+02:00 INFO [vulndb] Artifact successfully downloaded repo="mirror.gcr.io/aquasec/trivy-db:2"
adrian@DESKTOP-Q6JVA29: ~$
```

7: Descarga y actualiza la base de datos de vulnerabilidades

```
trivy image --download-db-only
```

Con esto, Trivy queda listo para analizar imágenes.

Caso práctico: escanear una imagen vulnerable y una limpia

Imagen vulnerable:

Vamos a usar una imagen antigua que suele ser más vulnerable que las actuales.

Por ejemplo, podemos coger python:3.7-slim, que es una versión relativamente antigua y puede contener vulnerabilidades en paquetes del sistema o incluso en Python mismo si no está actualizada.

Muchas veces las imágenes viejas se siguen usando en producción y traen CVEs.

La imagen python:3.7-slim es una imagen oficial de Docker que contiene Python 3.7, basada en una versión “slim” de Debian, nos da un entorno para ejecutar aplicaciones Python 3.7.

La versión “slim” significa que no trae herramientas de desarrollo ni utilidades extra, reduciendo el tamaño de la imagen

```
adrian@DESKTOP-Q6JVA29: ~$ sudo docker pull python:3.7-slim
[sudo] password for adrian:
3.7-slim: Pulling from library/python
a803e7c4b030: Pull complete
bf3336e84c8e: Pull complete
8973eb85275f: Pull complete
f9afc3cc0135: Pull complete
39312d8b4ab7: Pull complete
Digest: sha256:b53f496ca43e5af6994f8e316cf03af31050bf7944e0e4a308ad86c001cf028b
Status: Downloaded newer image for python:3.7-slim
docker.io/library/python:3.7-slim
adrian@DESKTOP-Q6JVA29: ~$
```

8: Pull python

```
$ sudo docker pull python:3.7-slim
```

Ahora haremos un escaneo de la imagen

```

adrian@DESKTOP-Q6JVA29:~$ trivy image python:3.7-slim
2025-10-19T20:01:10+02:00 INFO [vuln] Vulnerability scanning is enabled
2025-10-19T20:01:10+02:00 INFO [secret] Secret scanning is enabled
2025-10-19T20:01:10+02:00 INFO [secret] If your scanning is slow, please try '--scanners vuln' to disable secret scanning
2025-10-19T20:01:10+02:00 INFO [secret] Please see https://trivy.dev/v0.67/docs/scanner/secret#recommendation for faster secret detection
2025-10-19T20:01:12+02:00 INFO Detected OS family="debian" version="12.1"
2025-10-19T20:01:12+02:00 INFO [debian] Detecting vulnerabilities... os_version="12" pkg_num=106
2025-10-19T20:01:12+02:00 INFO Number of language-specific files num=1
2025-10-19T20:01:12+02:00 INFO [python-pkg] Detecting vulnerabilities...
2025-10-19T20:01:12+02:00 WARN Using severities from other vendors for some vulnerabilities. Read https://trivy.dev/v0.67/docs/scanner/vulnerability#severity
2025-10-19T20:01:12+02:00 INFO Table result includes only package filenames. Use '--format json' option to get the full path to the package file.

Report Summary
=====
+-----+-----+-----+-----+
| Target                                     | Type      | Vulnerabilities | Secrets |
+-----+-----+-----+-----+
| python:3.7-slim (debian 12.1)             | debian    | 234             | -       |
+-----+-----+-----+-----+
| usr/local/lib/python3.7/site-packages/pip-23.0.1.dist-info/METADATA | python-pkg | 2               | -       |
+-----+-----+-----+-----+
| usr/local/lib/python3.7/site-packages/setuptools-57.5.0.dist-info/METADATA | python-pkg | 3               | -       |
+-----+-----+-----+-----+
| usr/local/lib/python3.7/site-packages/wheel-0.41.2.dist-info/METADATA | python-pkg | 0               | -       |
+-----+-----+-----+-----+

Legend:
- '-': Not scanned
- '0': Clean (no security findings detected)

python:3.7-slim (debian 12.1)
Total: 234 (UNKNOWN: 0, LOW: 89, MEDIUM: 96, HIGH: 40, CRITICAL: 9)

```

9: Escaneo Imagen

```
$ trivy image python:3.7-slim
```

Podemos ver que nos da un sumario general de el numero de vulnerabilidades y de donde son,

El **Report Summary** muestra que la imagen tiene 234 vulnerabilidades en Debian 12.1, 2 en pip y 3 en setuptools y no se detectaron secretos.

La mayor parte del riesgo viene del sistema operativo, los paquetes Python tienen pocas vulnerabilidades.

Esto indica que se debería actualizar la base y los paquetes para reducir riesgos.

Además de esto da un print en la terminal de una tabla donde profundiza y detalla todo sobre las vulnerabilidades, una de las cosas que podemos ver es el nivel de riesgo:

- **CRITICAL**: actualizar base o parchear
- **HIGH**: evaluar riesgo según exposición y servicios
- **MEDIUM LOW**: documentar quizá no urgente

Podemos ver el Status Estado de la vulnerabilidad en la versión instalada, fixed arreglado, affected afecta la versión instalada, will_not_fix no se corregirá.

Entre otros detalles....

Library	Vulnerability	Severity	Status	Installed Version	Fixed Version	Title
apt	CVE-2011-3374	LOW	affected	2.6.1		It was found that apt-key in apt, all versions, do not correctly... https://nvd.nessus.com/nvd/cve-2011-3374
bash	TEMP-0841856-B18BAF			5.2.15-2+b2		[Privilege escalation possible to other user than root] https://security-tracker.debian.org/tracker/TEMP-0841856-B18BAF
bsdutils	CVE-2024-28085	MEDIUM	fixed	1:2.38.1-5+b1	2.38.1-5+deb12u1	util-linux: CVE-2024-28085: null: escape sequence inj... https://nvd.nessus.com/nvd/cve-2024-28085

10: Detalles de vulnerabilidades

Para manejar mejor estos datos podemos exportarlos a TXT y JSON, aunque hay posibilidad de otros formatos como:

- salida personalizada (puedes usar plantillas para HTML, Markdown, etc.)
- csv aunque no es nativo
- entre otros....

Con TXT obtenemos un reporte legible con tablas que se pueden revisar fácilmente:

```
$ trivy image -f table -o trivy-report.txt python:3.7-slim
```

Con JSON obtenemos un archivo estructurado que permite filtrar y procesar vulnerabilidades automáticamente

```
$ trivy image -f json -o trivy-report.json python:3.7-slim
```

Parámetros:

- -f <formato>
- -o <archivo>

Estos archivos se adjuntarán para ver el resultado de la salida de los mismos.

Imagen no vulnerable:

Vamos a usar la nueva versión de la imagen python:3.11-slim que es más reciente y segura y recibe actualizaciones de seguridad regularmente

```
adrian@DESKTOP-Q6JVA29:~$ sudo docker pull python:3.11-slim
[sudo] password for adrian:
3.11-slim: Pulling from library/python
8c7716127147: Pull complete
c72c56726626: Pull complete
76d93c681ade: Pull complete
80061c640d63: Pull complete
Digest: sha256:ff8533f48e12b705fc20d339fde2ec61d0b234dd9366bab3bc84d7b76a45c8c0
Status: Downloaded newer image for python:3.11-slim
docker.io/library/python:3.11-slim
adrian@DESKTOP-Q6JVA29:~$
```

11: pull python

```
$ sudo docker pull python:3.11-slim
```

Ahora haremos el escaneo:

Target	Type	Vulnerabilities	Secrets
python:3.11-slim (debian 13.1)	debian	52	-
usr/local/lib/python3.11/site-packages/pip-24.0.dist-info/METADATA	python-pkg	1	-
usr/local/lib/python3.11/site-packages/setuptools-79.0.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/autocommand-2.2.2.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/backports.tarfile-1.2.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/importlib_metadata-8.0.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/infect-7.3.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/jaraco.collections-5.1.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/jaraco.context-5.3.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/jaraco.functools-4.0.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/jaraco.text-3.12.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/more_itertools-10.3.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/packaging-24.2.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/platformdirs-4.2.2.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/tomli-2.0.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/typeguard-4.3.0.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/typing_extensions-4.12.2.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/wheel-0.45.1.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/setuptools/_vendor/zipper-3.19.2.dist-info/METADATA	python-pkg	0	-
usr/local/lib/python3.11/site-packages/wheel-0.45.1.dist-info/METADATA	python-pkg	0	-

12: resultado escaneo limpio

```
$ trivy image python:3.11-slim
```

La imagen es relativamente nueva, pero el escaneo muestra que todavía hay vulnerabilidades:

Total de vulnerabilidades:

Debian 13.1 : 52, la mayoría LOW, 1 MEDIUM, 0 HIGH o CRITICAL.

En paquetes de Python, solo tiene 1 MEDIUM (CVE-2025-8869) y está afectado, el resto está limpio.

La imagen está bastante segura comparada con versiones antiguas, pero aún hay que atender las vulnerabilidades y revisar si otras LOW requieren actualización.

Lo mismo que hemos hecho anteriormente , estas también las exportaremos y se subirán los archivos para verlos:

```
$ trivy image -f json -o resultado.json python:3.11-slim
$ trivy image -o resultado.txt python:3.11-slim
```

Integración de Trivy en CI/CD

CI/CD significa **Integración Continua y Despliegue Continuo**. Es un proceso automatizado donde cada cambio en el código se construye, prueba y despliega sin intervención manual, asegurando que todo funcione correctamente.

Un pipeline es la serie de pasos automatizados dentro de CI/CD que definen cómo se construye, prueba y despliega el proyecto;

En mi caso el pipeline de [GitHub Actions](#) va a construir la imagen Docker de mi proyecto Python, ejecutar Trivy para escanear vulnerabilidades y generar reportes .

Para empezar crearemos una estructura para simular un proyecto sencillo

```
# Crear estructura de carpetas
mkdir -p trivy-ci-demo/.github/workflows

# Crear archivos
touch trivy-ci-demo/app.py
touch trivy-ci-demo/requirements.txt
touch trivy-ci-demo/Dockerfile
touch trivy-ci-demo/.github/workflows/scan.yml
```

Figura 13: Crear estructura

Nos quedará esta estructura:

```
trivy-ci-demo/  
├── app.py  
├── requirements.txt  
├── Dockerfile  
└── .github/  
    ├── workflows/  
    └── scan.yml
```

```
# Crear estructura de carpetas  
mkdir -p trivy-ci-demo/.github/workflows  
  
# Crear archivos  
touch trivy-ci-demo/app.py  
touch trivy-ci-demo/requirements.txt (# Sin dependencias externas puede estar vacío)  
touch trivy-ci-demo/Dockerfile  
touch trivy-ci-demo/.github/workflows/scan.yml
```

Vamos a hacer una aplicación simple en Python, solo para tener algo que meter dentro de la imagen Docker.

```
adrian@DESKTOP-Q6JVA29:~/trivy-ci-demo$ cat app.py  
print("Aplicación de ejemplo para análisis con Trivy en CI/CD")  
adrian@DESKTOP-Q6JVA29:~/trivy-ci-demo$
```

14: Contenido app.py

Ahora escribimos el Dockerfile que define cómo se construye la imagen ,esto crea una imagen en Python 3.11

```
# Imagen base oficial de Python
FROM python:3.11-slim

# Establecer el directorio de trabajo
WORKDIR /app

# Copiar el archivo principal
COPY app.py .

# Comando que ejecutará el contenedor
CMD ["python", "app.py"]
```

15: Imagen Docker

```
# Imagen de Python
FROM python:3.11-slim

# Establecer el directorio de trabajo
WORKDIR /app

# Copiar el archivo principal
COPY app.py .

# Comando que ejecutará el contenedor
CMD ["python", "app.py"]
```

Continuamos con el siguiente archivo (.github/workflows/scan.yml), este archivo define el pipeline de GitHub Actions, es decir, lo que se ejecutará automáticamente cuando se suba algo al repositorio.

```

name: Trivy Scan

on:
  push:
    branches: [ main ]

jobs:
  scan:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Run Trivy Table Report
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: python:3.11-slim
          format: table
          output: results.txt
          exit-code: 0
          severity: CRITICAL,HIGH,MEDIUM,LOW
          ignore-unfixed: false

      - name: Run Trivy JSON Report
        uses: aquasecurity/trivy-action@master
        with:
          image-ref: python:3.11-slim
          format: json
          output: results.json
          severity: CRITICAL,HIGH,MEDIUM,LOW
          ignore-unfixed: false

      - name: Upload Trivy Artifacts
        uses: actions/upload-artifact@v4
        with:
          name: trivy-reports
          path: |
            results.txt
            results.json
  
```

```

name: Trivy Scan

on:
  push:
    branches: [ main ]

jobs:
  scan:
    runs-on: ubuntu-latest

    steps:
      - name: Checkout code
        uses: actions/checkout@v4

      - name: Run Trivy Table Report
        uses: aquasecurity/trivy-action@master
        with:
  
```

```
image-ref: python:3.11-slim
format: table
output: results.txt
exit-code: 0
severity: CRITICAL,HIGH,MEDIUM,LOW
ignore-unfixed: false

- name: Run Trivy JSON Report
  uses: aquasecurity/trivy-action@master
  with:
    image-ref: python:3.11-slim
    format: json
    output: results.json
    exit-code: 0
    severity: CRITICAL,HIGH,MEDIUM,LOW
    ignore-unfixed: false

- name: Upload Trivy Artifacts
  uses: actions/upload-artifact@v4
  with:
    name: trivy-reports
    path: |
      results.txt
      results.json
```

Esto automatiza el escaneo de la imagen cada vez que se suban cambios al branch main.

El nombre del workflow es Trivy Scan.

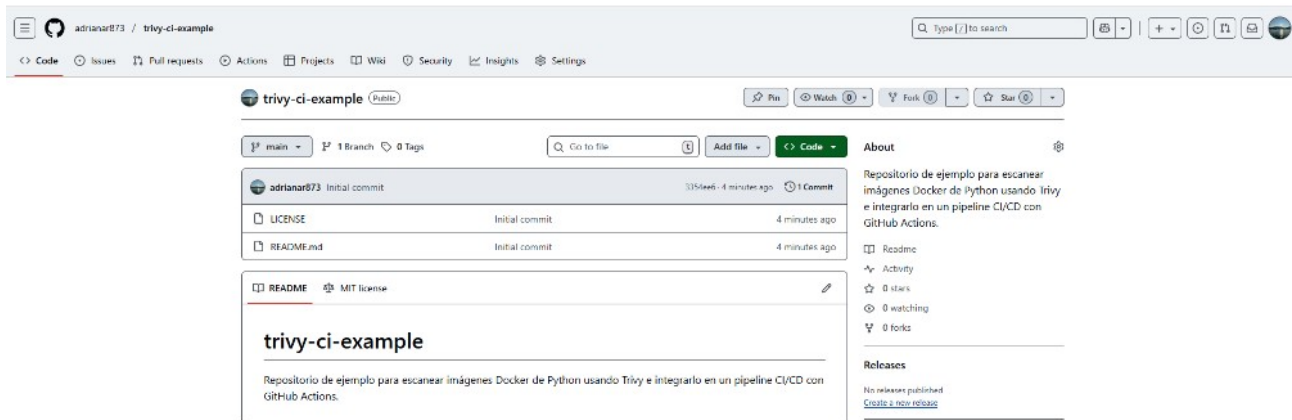
Se ejecuta cuando hay un push al branch main. Define un trabajo llamado scan que se ejecuta en una máquina Ubuntu .

Los pasos del jobs son: primero hace checkout del código del repositorio para poder usarlo, y luego escanea la imagen python 3.11 usando la acción oficial de Aqua Security.

El escaneo genera un informe en formato tabla y otro en json, lo guarda en results.txt,result.json, analiza todas las vulnerabilidades desde CRITICAL hasta LOW y no ignora las que no tienen solución.

Una vez rellenados estos archivos creamos el repositorio de GitHub:

Repo



16: Repositorio GitHub

Lo siguiente sería subir el proyecto

```

adrian@DESKTOP-Q6JVA29: ~$ git init
hint: Using 'master' as the name for the initial branch. This default branch name
hint: is subject to change. To configure the initial branch name to use in all
hint: of your new repositories, which will suppress this warning, call:
hint:
hint:   git config --global init.defaultBranch <name>
hint:
hint: Names commonly chosen instead of 'master' are 'main', 'trunk' and
hint: 'development'. The just-created branch can be renamed via this command:
hint:
hint:   git branch -m <name>
Initialized empty Git repository in /home/adrian/trivy-ci-demo/.git/
adrian@DESKTOP-Q6JVA29: ~$ git remote add origin https://github.com/adrianar873/trivy-ci-example.git
adrian@DESKTOP-Q6JVA29: ~$ git add .
adrian@DESKTOP-Q6JVA29: ~$ git commit -m "Commit proyecto Trivy"
Author identity unknown

*** Please tell me who you are:

Run

  git config --global user.email "you@example.com"
  git config --global user.name "Your Name"

to set your account's default identity.
Omit --global to set the identity only in this repository.

fatal: empty ident name (for <adrian@DESKTOP-Q6JVA29.>) not allowed
adrian@DESKTOP-Q6JVA29: ~$ git config --global user.email "adrian@example.com"
adrian@DESKTOP-Q6JVA29: ~$ git config --global user.name "Adrian"
adrian@DESKTOP-Q6JVA29: ~$ git commit -m "Commit proyecto Trivy"
[master (root-commit) 5a9a7c1] Commit proyecto Trivy
 4 files changed, 46 insertions(+)
 create mode 100644 .github/workflows/scan.yml
 create mode 100644 Dockerfile
 create mode 100644 app.py
 create mode 100644 requirements.txt
adrian@DESKTOP-Q6JVA29: ~$

```

17: Subir Proyecto

Por último finalizamos con subir los archivos al repositorio

```
adrian@DESKTOP-Q6JVA29:~/trivy-ci-demo$ git push -u origin main
Username for 'https://github.com': adrianar873
Password for 'https://adrianar873@github.com':
To https://github.com/adrianar873/trivy-ci-example.git
 ! [rejected]        main -> main (fetch first)
error: failed to push some refs to 'https://github.com/adrianar873/trivy-ci-example.git'
hint: Updates were rejected because the remote contains work that you do not
hint: have locally. This is usually caused by another repository pushing to
hint: the same ref. If you want to integrate the remote changes, use
hint: 'git pull' before pushing again.
hint: See the 'Note about fast-forwards' in 'git push --help' for details.
adrian@DESKTOP-Q6JVA29:~/trivy-ci-demo$ git push -u origin main --force
Username for 'https://github.com': adrianar873
Password for 'https://adrianar873@github.com':
Enumerating objects: 8, done.
Counting objects: 100% (8/8), done.
Delta compression using up to 12 threads
Compressing objects: 100% (5/5), done.
Writing objects: 100% (8/8), 1.00 KiB | 1.00 MiB/s, done.
Total 8 (delta 0), reused 0 (delta 0), pack-reused 0
To https://github.com/adrianar873/trivy-ci-example.git
+ 3354ee6...5a9a7c1 main -> main (forced update)
branch 'main' set up to track 'origin/main'.
adrian@DESKTOP-Q6JVA29:~/trivy-ci-demo$
```

18: Push al repositorio

El primer push falló porque el repositorio ya contenía contenido, por motivos de protección contra sobreescritura, Git no permitió subir los cambios directamente, así que fue se fuerza con push.

Una vez subido vamos al repositorio en GitHub y vamos a la pestaña de Actions y ahí podemos ver todos los logs y jobs ejecutados al hacer el push.

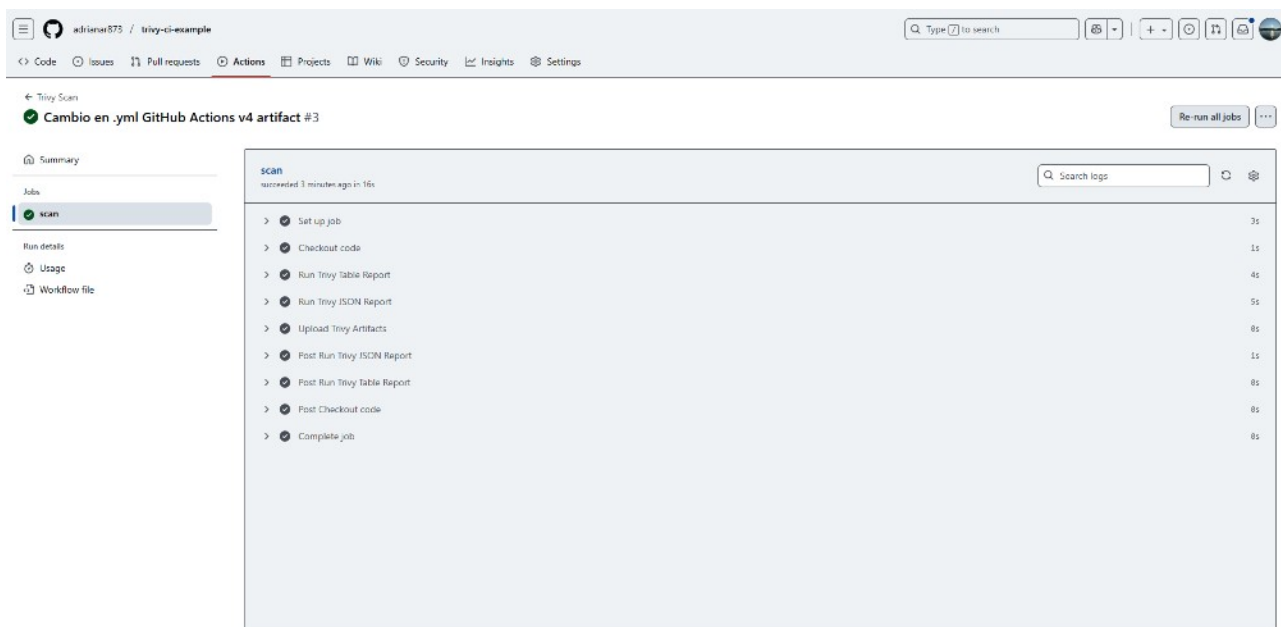


Figura 19: Logs Actions

Además de ver todos los logs podemos descargar los artifacts que es un .zip donde guarda el txt y json.

Summary

Jobs

scan

Run details

Usage

Workflow file

Triggered via push 5 minutes ago

adrianar873 pushed · 16s · main

Status

Success

Total duration

25s

Artifacts

1

scan.yml

src: push

scan

10s

Artifacts

Produced during runtime

Name	Size	Digest
trivy-reports	17.8 KB	sha256:f761ca8136d8c332920b7c7f5e6d8750ac04bccc494608c56078e581e6283de0752

20: Pestaña Actions

trivy-reports.zip

Este equipo > BIGGER (E:) > Nueva carpeta > trivy-reports.zip

Buscar en trivy-reports.zip

Nuevo

Ordenar

Ver

Extraer todo

Detalles

Nombre	Tipo	Tamaño comprimido	Protegido ...	Tamaño	Relación	Fecha de modificación
results.json	Archivo de origen JSON	14 KB	No	134 KB	90%	22/10/2025 20:07
results.txt	Documento de texto	4 KB	No	65 KB	94%	22/10/2025 20:07

21: Reports

Conclusiones

Durante este trabajo hemos comprobado la importancia de mantener la seguridad en entornos de contenedores y cómo las vulnerabilidades en imágenes y paquetes.

Exploramos Trivy, para detectar vulnerabilidades conocidas, generar reportes claros y exportarlos en formatos útiles como TXT yJSON.

Realizamos prácticas analizando imágenes antiguas y actualizadas, lo que permitió ver cómo las actualizaciones reducen riesgos y la forma de interpretar los resultados: severidad, estado y versiones afectadas o corregidas...

Además, integramos Trivy en un pipeline de CI/CD con GitHub Actions, automatizando el escaneo de imágenes en cada push y asegurando un flujo continuo de seguridad.

En general, este trabajo mostró que la combinación de buenas prácticas de actualización de dependencias, herramientas de escaneo como Trivy y automatización en CI/CD permite mantener entornos de desarrollo y producción más seguros y controlados, facilitando la detección temprana y la gestión de riesgos de manera eficiente.